

数组张量与形状思维实验

目标与准备

把普通Python循环改写为NumPy数组运算，并逐项证明它在本例上没有改变含义。再构造一次不报错的错误广播，解释为什么它把三项误差变成九项。输出包括每个中间数组的形状、类型和数值，以及逐条预测记录。

先修003。CPU即可，实验不下载数据、不调用API。需要Python3.12和NumPy2.3系列；课程执行使用NumPy2.3.5。环境文件列出建议版本和Notebook依赖，尚未声称每种操作系统的安装都已测试。

已有课程独立环境时可在已激活的环境中安装NumPy。使用官方包渠道，并按实际提示确认安装；不要为解决包问题关闭系统安全设置。命令示例为 `conda install -c conda-forge numpy=2.3.5`。若已在别的受管理环境运行，先查看现有版本，避免未经计划修改其他项目环境。

一 先填形状表

从data/calibration.csv抄下三条输入。按第一列旋钮、第二列旋钮的顺序创建X，写出X(3,2)的样本轴和特征轴。w包含2和0.1，形状(2,); y包含5、7、9，形状(3,)。标签不进入固定预测公式。

先不运行代码，写下weighted、combined、predictions和errors的预期形状及数值。weighted应为 [[2,1],[4,2],[6,3]]，combined为 [3,6,9]，predictions为 [4,7,10]，errors为 [-1,0,1]。绝对误差总和2除以3，得到约0.667。

本单元不拟合参数。数据用于演示轴与计算，不是新的模型比较研究。没有训练或测试划分，不应把固定规则在三条样本上的MAE包装成泛化分数。

二 两条计算路径逐项核对

experiment.py提供loop_predict与array_predict。前者把每行转换成普通数列表，依次计算两个乘积和偏移量；后者先做 `X * weights`，再 `sum(axis=1)`，最后加1。测试不只比较最终平均，还对照每个中间数组。

在本目录执行：

```
python experiment.py
python test_experiment.py
```

第一个命令生成outputs/array_ledger.json，按名称保存shape、dtype和全部值；predictions.csv保留编号、标签、预测、绝对误差；metrics.json保存正确MAE和明确标为错误示范的两两平均；environment.json保存版本。再次运行更新同名输出，原始CSV不变。

第二个命令执行八组测试，覆盖逐项手算、轴归约、广播故障、形状与有限值检查、矩阵乘法、reshape、视图与24个通道编号。正常结束应打印status为passed。不要只数通过组数，要知道每组排除了什么错误。

三 主动制造九个配对

在Notebook中创建一维predictions和labels，先查看两者shape，再计算正确误差。随后只把预测改为 `predictions.reshape(3,1)`，标签不变。运行减法后立即打印结果shape，应为(3,3)，而不是预期的(3,)或(3,1)。

手工逐行核对：第一行-1、-3、-5，第二行2、0、-2，第三行5、3、1。九项绝对值加起来22，除以9约为2.444。读取array_ledger中的pairwise_errors_deliberate_bug，确认它被明确标成故障示例，不能混入正式逐样本指标。

现在把列形状预测交给strict_mae，应该抛出ValueError。Notebook会捕捉这一预期异常并打印说明。函数拒绝了形状不匹配，说明错误被发现；它没有自动猜测你是想做逐样本评价还是两两距离研究。

四 改变axis并说出意义

对weighted分别沿axis=0和axis=1求和。前者得到(2,)的 `[12,6]`，表示每个特征跨三条样本的总贡献；后者得到(3,)的 `[3,6,9]`，表示每条样本的总贡献。然后加keepdims=True，得到(3,1)，确认轴位置保留但长度为1。

对weighted直接mean得到3，是六项贡献的总体平均。它不是误差，因为这一步没有使用标签。写报告时应把每个数放回它的定义，不要把所有平均都写作loss。

五 区分矩阵乘法与形状重排

创建W为 `[[2,-1],[0.1,0.2]]`，手算每行与每列的乘积和。X@W应为 `[[3,1],[6,2],[9,3]]`；加偏移 `[1,-1]`后为 `[[4,0],[7,1],[10,2]]`。@执行配对乘积后求和，*则逐元素相乘并应用广播规则。

分别打印X.reshape(2,3)和X.T。前者为 `[[1,10,2],[20,3,30]]`，后者为 `[[1,2,3],[10,20,30]]`。它们形状相同，但不逐项相等。用 `np.array_equal`检查准确相等；对需要小数近似的计算，先检查shape，再用 `np.testing.assert_allclose`并说明容差。

不要用一次容许广播的近似比较代替shape检查。对于本课程小数组，绝对容差取1e-12、相对容差取0；这只适合当前数值尺度和float64计算，并非所有深度模型的通用标准。

六 用副本演示视图

先 `isolated = X.copy()`，再 `view = isolated[:,0]`，把view第一项设成99。此时isolated第一行第一列也变成99，说明基本切片共享存储。原X不变，因为最开始复制了整份输入。

再将切片`.copy()`保存为`independent`，修改`independent`第一项，`isolated`不应再变化。这个演示说明新名字不等于新数据，显式副本应在需要独立存储时建立。不要在正式原始数组上随意做演示，否则后面的对照实验会被前面的操作污染。

七 追踪每个通道元素

`channel_marker`构造(2,3,2,2)数组。位置`n`、`c`、`h`、`w`存入 $1000*n + 100*c + 10*h + w$ ，每个允许位置都有不同编号。例如(1,2,0,1)存1201。由NCHW变成NHWC使用 `transpose(0,2,3,1)`，意思是新轴依次取原来的第0、2、3、1轴。

变换后`shape`为(2,2,2,3)，编号1201应在(1,0,1,2)。测试逐个核对全部24个位置，而不是只看总和。总和在错误排列后也可能完全相同，不能证明通道对应正确。

八 完成条件

保留原手算表、`array_ledger`、逐条预测、八组测试结果和一段广播故障说明。解释你每次`reshape`或`transpose`的轴含义，而不是只说“这样能运行”。打开Notebook后重启内核并顺序执行全部，输出保存到`notebook_outputs`。

制作时实际执行方式是全新Python进程中的真实IPython `InProcessKernel`；未验证浏览器Jupyter界面或外进程套接字传输。小样例只检查计算与数据对应，不构成性能或泛化结论。练习完整答案见`answers.pdf`。